

LAB : *Multi-Agents en Langchain*

Table des matières

PARTIE 1 : Définition des outils	2
PARTIE 2 : Création des sous agents	2
PARTIE 3 : Créer l'agent principal.....	3
PARTIE 4 : Appeler les agents et afficher le résultat	3
PARTIE 5 : Utiliser LangGraph Studio pour simuler les 3 agents	4

PARTIE 1 : Définition des outils

```
from langchain.tools import tool

@tool
def square_root(x: float) -> float:
    """Calculate the square root of a number"""
    return x ** 0.5

@tool
def square(x: float) -> float:
    """Calculate the square of a number"""
    return x ** 2
```

PARTIE 2 : Création des sous agents

```
from langchain.agents import create_agent
from langchain_ollama import ChatOllama

model = ChatOllama(
    model="llama3.2:3b",
)

subagent_1 = create_agent(
    model='gpt-5-nano',
    tools=[square_root]
)

subagent_2 = create_agent(
    model='gpt-5-nano',
    tools=[square]
)
```

PARTIE 3 : Créer l'agent principal

```
@tool
def call_subagent_1(x: float) -> float:
    """Call subagent 1 in order to calculate the square root of a number"""
    response = subagent_1.invoke({"messages": [HumanMessage(content=f"Calculate the square root of {x}")]})
    return response["messages"][-1].content

@tool
def call_subagent_2(x: float) -> float:
    """Call subagent 2 in order to calculate the square of a number"""
    response = subagent_2.invoke({"messages": [HumanMessage(content=f"Calculate the square of {x}")]})
    return response["messages"][-1].content

main_agent = create_agent(
    model='gpt-5-nano',
    tools=[call_subagent_1, call_subagent_2],
    system_prompt="You are a helpful assistant who can call subagents to calculate the square root or square of a number.")
```

PARTIE 4 : Appeler les agents et afficher le résultat

```
question = "What is the square root of 456?"

response = main_agent.invoke({"messages": [HumanMessage(content=question)]})
print(response['messages'][-1].content)
```

PARTIE 5 : Utiliser LangGraph Studio pour simuler les 3 agents .

Configurer le fichier langgraph.json afin de visualiser et tester les 3 agents .

The screenshot displays the LangSmith Studio interface for configuring and testing a LangGraph workflow. The left sidebar shows the application navigation menu, including options like Search, Home, Tracing, Monitoring, Datasets & Experiments, Annotation Queues, Prompts, Playground, Studio (selected), Deployments, and Sandboxes. The main workspace is divided into two panels: 'Graph' and 'Chat'.

The 'Graph' panel shows a workflow diagram with three nodes: `__start__` (grey), `model` (green), and `__end__` (grey). A yellow node labeled `tools` is connected to the `model` node. The workflow starts at `__start__`, goes to `model`, then to `__end__`, with a loop back from `tools` to `model`.

The 'Chat' panel shows a simulation of the workflow. It includes a 'Thread' identifier, a 'Turn' counter (TURN 1), and buttons for 'Interact', 'Trace', and 'Run experiment'. The chat history shows the following interactions:

- Human:** "What is the square root of 456?"
- Model:** "The square root of 456.0 is approximately 21.3541565 (≈ 21.3542 rounded to 4 decimal places)." (This response is shown as a callout from the `tools` node).
- AI:** "The square root of 456 is approximately 21.3541565, which rounds to 21.3542 to four decimal places."

The bottom of the interface features an 'Input' section with a 'Messages' list, a '+ Message' button, and a 'Submit' button.